Python:- It is an Interpreted language.

Ⓐ Python also have compiler, which identifies Syntax Error

→ methods in python (using dir in jupyter):

HW ① Different roles related to this field, their responsibilities, skills required, diff.

② methods in each data structures: String, list, tuple, frozenset, .

③ when do we use list and tuple?

④ Basics of python

H.W Methods in each data structure:

(I.) String Methods:-

	use	example
① upper()	converts to upper case	"hello".upper() o/p: "HELLO"
② lower()	" " lower case	"HELLO".lower() o/p:- "hello"
③ capitalize()	First letter uppercase	"hello".capitalize() o/p: "Hello"
④ title()	First letter of each word is upper case	"Ravi is a boy".title() o/p: Ravi Is A Boy
⑤ swapcase()	swaps upper/lower case	"pYtHoN".swapcase() o/p- PyThOn.
⑥ strip()	Remove spaces from both ends	" hello ".strip() o/p- "hello"
⑦ lstrip()	Remove left spaces only	" hello ".lstrip() o/p: "hello "
⑧ rstrip()	Remove Right spaces only	" hello ".rstrip() o/p: " hello"
⑨ replace()	replaces particular substring with given character in all occurances.	"hello".replace("x", "x") o/p: "hexxo"

⑩ split()	split string into list (Default is " " space)	"a,b,c".split(",") o/p: ["a","b","c"]
⑪ join()	join list of elements into string using specified operator.	"-".join(["a","b","c"]) o/p: "a-b-c"
⑫ find()	find first occurance of a specific substring (not give error, it give "-1" if not found)	"Python".find("t") o/p: 2
⑬ index()	finds index for a character (error if it is absent).	"python".index("y") o/p: 1
⑭ count()	count occurances of a specified substring.	"banana".count("a") o/p: 3.
⑮ starts with()	checks string starting text is specified prefix.	"Python".startswith("Py") o/p: True
⑯ ends with()	checks ending of string is specified suffix.	"Python".endswith("on") o/p: True
⑰ isalpha()	To check only letters should be there in string (& not digits, spaces, symbols)	⑰ "Python".isalpha() o/p: True. ⑤ "Python 3".isalpha() o/p: False
⑱ isdigit()	Only digits (0-9), superscript (², ³).	
⑲ isalnum()	True, Only letter & digits should be in string.	Because, space is not alphabets.

isdigit()

"123".isdigit() o/p: True

"123 ".isdigit() o/p: false (space is also true)

isalnum() not an alphabet (&) number

"abc123".isalnum() o/p: True

"@b1".isalnum() o/p: False

→ (II.) List Methods:-

① append()	Adds one element	lst.append(5)
② extend()	Adds multiple elements.	lst.extend([6,7])
③ insert()	Inserts at index specified.	lst.insert(1,10) Index element
④ remove()	Remove first matching value	lst.remove(10)

sort, also used for descending order,

lst.sort(reverse = True)

5. pop()	Remove by index (default last)	lst.pop() [Remove last element] / lst.pop(3) [3rd index element]
6. clear()	Removes all elements from list & give empty list	lst.clear()
7. index()	finds index for element (if no element, Value error occur)	lst.index(5) [5 = element]
8. count()	Count occurances of specific element	lst.count(5)
9. sort()	sort in ascending order	lst.sort()
10. reverse()	Reverses order	lst.reverse()
11. copy()	creates shallow copy [doesn't modifies original list, when we update shallow copy]	new list = lst.copy()

modifies original list & modify original list

(III.) Tuple methods:- If is separate list. t = (1,2,3,2,4)

1. count() - count occurances of specific element - t.count(2)
2. index() - Finds index for element - t.index(2)

(IV.) Set methods:- If set a = {1,2,3}

1. add()	Adds one element	Set_a.add(3) o/p: {1,2,3,4}.
2. update()	Adds multiple items to the set	Set_a.update([3,5,6]) o/p: {1,2,5,6,3}.
3. remove()	Removes element (Key error if absent)	Set_a.remove(2)
4. discard()	Removes element safely (Gives same set, if element absent)	set_a.discard(2) → doesn't throw error
5. pop()	pop remove Random element	s.pop()
6. clear()	Remove all Elements	s.clear() o/p: set()
7. copy()	creates a copy (original set doesn't modify)	new_s = s.copy()

The following methods are common b/w sets & frozen sets.

→ As frozen sets are immutable Add(), update(), update(), discard() is not available.

(V.) Frozen sets:- unchangeable collection of unique items. It is immutable (you can't add, remove, update etc)

eg: x = frozenset(["a","b","c"])
print(x) o/p: frozenset({'a','b','c'})

x.add("d") => Error (you can't add element).

fs1 = frozenset

Symbols

(|)	(8.) union()	combine sets & combine frozensets	s1.union(s2) / fs1.union(fs2)
(&)	(9.) intersection()	common elements	s1.intersection(s2) / fs1. " (s2)
(-)	(10.) difference()	elements in first set only	s1.difference(s2) / fs1. " (s2)
(^)	(11.) symmetric_difference()	No common elements	s1.symmetric_difference(s2) / fs1. " (fs2)
Return new set	(12.) issubset()	is first set is subset of second set	s1.issubset(s2) / fs1. " (fs2)
	(13.) issuperset()	" " " superset of "	s1.issuperset(s2) / fs1. " " (fs2)
	(14.) isdisjoint()	checks no common elements	s1.isdisjoint(s2) / fs1. " (fs2)
T(0) as o/p	(15.) copy	return same frozenset	fs.copy().

(VI.) Dictionaries:-

1. get()	get value by key (Value if Key not available)	d.get("name")
2. keys()	Return all keys	d.keys().

3. values()	Return all values	d.values()
4. items()	" all key-value pairs	d.items()
5. update()	update dictionary	d.update({"age":20})
6. pop()	Removes key	d.pop("name")
7. popitem()	Remove last inserted item	d.popitem()
8. setdefault()	~~sets~~ gets value (0) & inserts default	d.setdefault ("city", "Hyd").
9. copy()	creates shallow copy	new_d = d.copy()
10. clear()	Remove all items	d.clear()
11. fromkey()	create dictionaries from keys.	dict.fromkeys ([a, b, c]).

⇒ Use a list when data needs to be frequently modified like adding, updating etc...
Eg:- Shopping carts, student lists.

⇒ use a TUPLE when data is fixed and should not change.
Eg: RGB color values, Date of birth

=

-07-2026

**

→ (2) casefold(): Converts the string to lower case more aggressively than lower().

→ used for case insensitive comparissions.

Both lower() and casefold() convert a string to lower case, casefold() is especially used for International (unicode) text.

(1) Eg:- Normal English

```
text = "HELLO"
print(text.lower())
print(text.casefold())
```

o/p:
hello
hello.

(2) "German ß"

```
text = "Straße"
print(text.lower())
print(text.casefold())
```

o/p:
straße (lower() keeps ß as unchanged)
strasse
(converts ß → ss, making comparission more reliable)

(3) Case insensitive comparissions:-

```
s1 = "Straße"
s2 = "STRASSE"
print(s1.lower() == s2.lower())
print(s1.casefold() == s2.casefold())
```

o/p: False
True
[As, "straße" ≠ "strasse"
"strasse" = "strasse"]

→ use lower() when: converting text for display, working mainly with english text.

→ use casefold() when: performing case-insensitive comparisson Handling multilingual / unicode text.

→ (3) center():- Returns a centered string "within" the specified width

Eg:-
```
text = "AI"
print(text.center(8, "*"))
```
o/p: ***AI***
8 characters width.

Note:- If "odd number of spaces left", it will fill left side first.

```
print(text.center(9,"*"))
```
o/p: ****AI***
[AI → takes 2 character length, for Remaining 7 character 4 on left, 3 on Right]

→ (4) count()

→ (5) encode():- converts a string (str) into bytes using a specified encoding (UTF-8 by default).

Eg:
```
text = "Hello"
result = text.encode()
print(result)
print(type(result))
```
o/p:
b'Hello'
<class 'bytes'>

also written as: result = text.encode(encoding='utf-8', errors='strict')

encoding (optional): default: "utf-8"

errors (optional):
① "strict" (default) → Raise an error
② "ignore" → ignore invalid characters
③ "replace" → replaces invalid characters

Eg:- ① text = "café"
result = text.encode("utf-8")
print(result)
o/p: b'caf\xc3\xa9' ['é' is stored as multiple bytes in UTF-8]

decode() ⇒ converts bytes to string.

```
text = "Python"
encoded = text.encode()
decoded = encoded.decode()
print(encoded)
print(decoded)
```
o/p: b'Python'
Python

→ (6.) ends with() → Returns True (or) False.

→ (7) expand tabs() :- Replaces tab characters (\t) with spaces.

In general it" give 4 spaces.
By using expand tabs(), we can improve spaces
[By defaut 8 spaces, we can customize as we need]

Eg:- text = "Python \t AI"

	o/p:
print (text)	Python AI.
print (text.expandtabs()) default=8	Python AI } same
" (" " (8))	Python AI }
" (" " (20))	Python AI

★★ Eg:- (2) text = "Py \t Programming"

	o/p:
print (text.expandtabs(6))	'Py Programming' (6 spaces)
" (" " (2))	'Py Programming'
" (" " (4))	'Py Programming'

Explanation:-
① "py" has 2 characters.
② expand tabs(8) → next stop is 8 → 6 spaces.
③ " " (2) → already tab stop.
④ " " (4) → next tab space is 4 → 2 spaces.

No. of spaces = tabsize - (current position % tabsize)

→ (8.) Find() ; Return "-1" if substring is not found.
→ (9.) Index() ; "Value Error" is Raised, if substring not found.

Both, Returns the index of first occurrance of the specified substring

→ (10) format() :- Formats a string by replacing placeholders ({}) with values.

text = "My Name is {}. I am {} year's old".
print (text.format ("Ramesh", 24))

o/p:- My Name is Ramesh. I am 24 year's old.

→ (11.) format_map() :- Formats a string using values from a dictionary.

text = "Name is {name}, Age is {age}"
dict = {"name": "Ramesh", "age": 24}.
print (text.format_map (dict))

o/p:- Name is Ramesh, Age is 24.

→ (12.) isalnum()
→ (13.) isalpha()
→ (14.) isascii() :- Returns 'True', if all characters in the string are ASCII characters (0-127)

Eg:- text = "Python"
print (text.isascii())
o/p:- True

Eg:- text = "Pythön"
print (text.isascii())
o/p:- False (As, ö is not an ASCII value).

→ (15) isdecimal() :- Return 'True' if all characters are decimal digits (0-9)

Eg:- text = "123"
print (text.isdecimal())
o/p:- True

Eg:- text = "12.5"
o/p:- False.
As, . is not decimal digit

④ eg text = "12-5"

Eg:

① Normal	② Superscript Digits	③ Fraction
text = "123"	text = "² ³"	text = "½" (0) "Ⅴ"
print(text.isdigit()) →	→	→
print(text.isnumeric()) →	→	→
o/p:- True	o/p: True	o/p: False
True	True	True.

False, False } for both.

isdecimal() C isdigit() C isnumeric()

superset ↓ superset

→ (20.) isprintable():- Returns True if all characters are printable

Eg:

①	②
text = "Hello"	text = "Hello\n"
print(text.isprintable()) →	→
o/p: True	o/p: False (as \n doesn't print).

→ (21.) isspace():- Return True if all characters are white spaces

Eg:

①	②
text = " "	text = " a"
print(text.isspace()) →	→
o/p: True	o/p: False.

→ (22.) istitle :- Return True if ~~all~~ every word in string, start with capital (upper case) letter.

a = "Ram Is A Boy"	a = "Ram is a boy"
a.istitle() →	→
o/p: True	o/p: False.

→ (23.) isupper:- Return True, if all characters are upper case.

a = "RAMESH"	a = "Ramesh"
a.isupper() →	→
o/p: True	o/p: False.

(29.) translate():- Replace characters using a translation table created by maketrans()

Eg:-
```
table = str.maketrans("abc","123")
text = "abcxyz"
print(text.transform(table))
```
O/P:- 123xyz

Another Example:
```
table = str.maketrans("aeiou", "12345")
text = "education"
print(text.translate(table))
```
O/P:- 2d5c1t34n

Translation Table:

a	→	1
e	→	2
i	→	3
o	→	4
u	→	5

→ table=str.maketrans("aeiou", "") (Empty string)
O/P:- dctn → Remaining are characters are Replaced with empty string.

simply;
maketrans() → creates secret code table.
translate() → uses the table to convert the text.

(I.Q.)

replace()	translate()
(1) Replace one substring/character at a time.	(1) Can replace many different characters in single operation.
(2) used for small replacements.	(2) more efficient for multiple character mappings.

Eg:
```
text = "abcabc"
print(text.replace("a","1"))
```
O/P:- 1bc1bc

Eg:
```
table = str.maketrans("abc","123")
text = "abcabc"
print(text.translate(table))
```
O/P:- 123123

→ (30) partition():- splits the string into 3 parts at first occurance of the separator specified.

Eg:
```
text = "apple-banana-orange"
print (text.partition("-"))
```
o/p: ('apple', '-', 'banana-orange')

Note:- we need to specify the argument in partition, otherwise it will throw an error.

→ If we specify an operator, not from string, it will divide entire string as one part & 2 empty strings of another part.

Eg:
```
text = "apple-banana-orange"
print (text.partition("*"))
```
o/p: ('apple-banana-orange', '', '')

→ (31) removeprefix():- Removes the specified prefix, if exists.

Eg:
```
text = "unhappy"
print (text.removeprefix("un"))
```
o/p: happy

→ (32) removesuffix():- Removes the specified suffix, if exists.

Eg:
```
text = "Notes.txt"
print (text.removesuffix(".txt"))
```
o/p: Notes

Note:- If given substring doesn't have, it will give entire string instead of error.

→ (33) replace()

→ (34) rfind():- Returns the last occurance index of a substring. Returns "-1" if not found.

```
text="aba"; print (text.rfind("a"))
```
o/p: 2

("f")
o/p: -1

→ (35) rindex():- Returns the last occurance of a substring. But it gives value error, if substring not found.

Eg: text = "aba"; text.rfind("a"); text.rfind("f");
o/p: 2 | o/p: -1 Value error.

→ (36) rjust():- Right aligns the string & fills the remaining width with specified character.

Eg:
```
text = "AI"
print (text.rjust(6, "*"))
```
o/p: ****AI

→ (37) rpartition():- splits the string into 3 parts at the last occurance of the separator.

Eg:
```
text = "a-b-c"
print = text.rpartition("-")
```
o/p: ('a-b', '-', 'c')

→ (38) rsplit():- splits string from right side.

Eg:
Very similar to split(), But maxsplit make diffence
↓
maximum no. of times we need to split the substring.

Eg:
```
text = "a,b,c,d,e,f,g,h"
print (text.split(",", 2))
print (text.rsplit(",", 2))
```
(Right to left)

o/p: ['a,b,c,d,e,f', 'g', 'h']
remaining string 2 1

split from left to right, for the first two characters, with "," as separator. It will split(2) remaining can't not be split.

o/p: ['a', 'b', 'c,d,e,f,g,h']
1 2 Remaining string.

Normally without max split:
```
print (text.split(","))  → o/p: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
print (text.rsplit(","))  → ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

→ (39) rstrip() → (40) split() → (41) startswith()
→ (42) strip() → (43) swapcase() → (44) title()
→ ~~(45)~~ to (45) upper().
→ (46) Splitlines():- splits the string at linebreaks.

```
text = "Python\nJava\nC++"
print (text.splitlines())
```
o/p:- ['Python', 'Java', 'C++'].

→ (47) zfill():- Pads the string with leading zeroes until it reaches the specified length.

Eg:-
```
text = "25"
print (text.zfill(5))
```
o/p:- 00025

(III) Set Methods:- While performing operations like, difference, intersection, union, symmetric difference, it returns new set.

But;

(1) difference_update():- Removes elements from original set (set 1) that are not from set 2.

set1.difference(set2) → Return new set with only unequal values from set 1.
set1.difference_update(set2) → set 1 [removes the common elements with set 2].
o/p:-

(2) intersection_update():- Keeps only common elements in the original set.

```
Set_A = {1,2,3,4}   Set_B = {2,3}
Set_A.intersection_update(Set_B)
Print(Set_A)
```
o/p:- {2,3}

→ Suppose for intersection, original set doesn't change, new set created.

(3) Symmetric_difference_update():- Updates original set with elements present in either set, but not both.

```
A = {1,2,3}   B = {3,4,5}
A.Symmetric_difference_update(B)
print(A)
```
o/p:- {1,2,4,5}.

→ File:- A file is a container used to store data permanently on a storage device

Eg:- Student.txt, image.jpg, song.mp3 | movie.mp4, data.csv etc..

These files are not removed even after program closes.

Eg:- Name = "Ramesh"
After program closes, print(Name) o/p:- Name not defined.

Uses:- when we create student Management system, ATM, Bank applications, social media etc..,
we store ID's, passwords, photos, videos

2 categories:-

(1) Text files.
Eg:- .txt, .xml, .csv, .py, .json, .html, .css.
we can read it

(2) Binary files. (Store as bytes).
.jpg, .mp3, .mp4, .pdf, .exe,
→ we can't read it, we use files.

Mode :- Tell Python, how to open the file.
→ used only when opening the file.
→ specified inside open()
→ eg: r, w, a, r+, w+, a+, x

(*) File Handling :- It is a process of creating, opening, Reading, writing, updating, appending, Deleting, Renaming, using python file

(1) opening a file :- `file = open (filename, mode)`

eg :- `file = open("student.txt", "r")` ← read mode

`print (type (file))`

o/p :- <class '_io.TextIOWrapper'> } file object.

(*)(*) Modes :-

r → read	r+ → Read + Write
w → write	w+ → Write + Read
a → append	a+ → append + Read
x → create	

rb → read binary
wb → write binary
ab → append binary

Simply, when we open a an existing file, we can read any time.

→ (i) Read Mode (r) :- used only for Reading.

eg :-
```
file = open("student.txt","r")
print (file.read())
file.close()
```

student.txt
Python
Java
C++

o/p :-
Python
Java
C++

If file not exists, we get File Not Found Error

→ (ii) Write Mode (w) :- write data. If file exits, it will be ↓ Delete everything, then write new content.

Python creates

Method :- Tell Python, which operation to perform on the opened file.
→ used after file been opened.
→ read(), write(), append(), close(), seek(), tell().

eg :-
```
file = open("student.txt","w")
file.write("C++","Net")
file.close()
```

Now; student.txt is
C++
Net

As already we have file, it Re writes.

→ (iii) append Mode (a) :- Add data to End of existing data.

```
file = open("student.txt","a")
file.append("\n Java")
file.close()
```

Before :- Python
after :- Python
Java

→ (iv) create (x) :- create new file. If already file exists, error exists.

`file = open("abc.txt","x")` ← new file name

→ (v) Read & write (r+) :- `file = open("student.txt","r+")`
can write as well, but can't create new

→ (vi) write & Read (w+) :- If file exists → erase Content of everything, write
If not exists → create new file.

→ (vii) Append & Read (a+) :- writes at end, can also read, create new file if missing.

→ rb, wb, ab used for Binary Images, Videos, Audio, Pdf etc.
`file = open("img.jpg","rb")`

(*)(*) Methods :-

student.txt :- Python
Java
C++

(1) read() → Reads entire file (eg, given in Read Mode).
(2) read (size) → Reads only given characters.

eg :-
```
file = open("student.txt")
print (file.read(6))
```
→ only 6 characters are read.

o/p :- Python

```
file.read(9)
```
o/p :- Python Ja

(3) readline() → Reads only one line in file.

Eg:- Python
Java
C++
.net

```
file = open("student.txt")
print(file.readline())
print(file.readline())
```

o/p:- Python
Java

(Each call reads a new line).

(4) readlines() → Returns list.

```
print(file.readlines())
```

o/p:- ['Python\n', 'Java\n', 'C++']

** (5) write() :- write a string in to file.

```
file = open("student.txt", "w")
file.write("Python")
file.close()
```

o/p:- Python

(6) writelines() :- write multiple strings in to file.

```
file.writelines(["Python", "\nJava", "\nC++"])
file.close()
```

o/p:- Python
Java
C++

Note:- Pointer movement in Reading.

```
file = open("student.txt")
print(file.read(6))
print(file.read())
```

o/p:- Python
programming

student.txt
Python programming

Read 6 characters, Now pointer is at 7th index, just before Programming. Read from, space, Program, so it gives, programming
space.

(7) tell() :- Returns the current position of the file pointer.

Eg:-
```
file = open("student.txt", "r")
print(file.tell())
file.read(5)
print(file.tell())
file.close()
```

o/p:- 0 [As, it haven't started Reading].
5 [As, it Read 5 characters, before this].

(8) seek() :- Moves the file pointer to specified position (index)

Eg:-
```
file = open("student.txt", "r").
file.seek(6)
print(file.read())
file.close()
```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
Python programming

o/p:- programming.
space also taken

(9) close() :- If we don't close the file, it can cause data loss, bugs, system instability.

```
file.close()
```

(10) With statment :- Best way to open file, because it will automatically closes file.

Eg:- Normal
```
file = open("student.txt", "r")
print(file.read())
file.close()
```

With statment:-
```
with open("student.txt", "r") as file:
    print(file.read())
```

(11) encoding :- Default:- utf-8

```
with open("student.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

Needed for:- Telugu,
Hindi,
Chinese, Russian, etc...

(iii) For Binary files

Reading image	Copy image
with open("photo.jpg", "rb") as file: data = file.read()	with open("n.jpg", "wb") as file: file.write(data).

(iv) Exception handling:-

```
try:
    with open("student.txt") as file:
        print(file.read())
except FileNotFoundError:
    print("File Not Found")
```

(4) OS File operations:-

(1) Import:- import os

(2) Check file exists:- os.path.exists("student.txt")

True (if exists) | False (if not exists)

(3) Delete file:-

os.remove("student.txt")

(4) Rename file:- os.rename("old.txt", "new.txt")

(5) Create folder / Directory:- os.mkdir("Python")

(6) Remove folder / Directory:- os.rmdir("Python")

(5) JSON (Java script object Notation):-

light weight, data-interchange format used to store & exchange data b/w applications.

→ Easy to write, human readable, easy to parse.
→ used to store structured data,
Exchange data b/w client & server.

Eg:- {
"name" : "Ramesh",
"age" : 22,
"course" : "AI/ML",
"skills" : ["Python", "SQL", "Machine Learning"]
"is_student" : true
}

Note:- true, false, null, object is written text format, only use double quotes.

(1) Import :- import json

(2) JSON Methods:- (4)

(1.) dump() :- writes a python object to a JSON file.

Syntax :- json.dump(object, file)

Eg:-

```
import json
student = {
    "name": "Ramesh",
    "age": 22
}
with open("student.json", "w") as file:
    json.dump(student, file).
```

(Python object)

o/p
{"name": "Ramesh", "age": 22}

Note:- Creates a file, if not exists.

→ (2) dumps() :- Converts a python object into a JSON string.

→ Output is a string, not a file.

```
eg:- import json
     student = {
       "name": "Ramesh",
       "age": 22
     }
     result = json.dumps(student)
     print(result)
```

o/p:-
{"name": "Ramesh", "age": 22}

→ (3) load() :- Reads JSON data from a file & converts it to a python object.

→ Return Python dictionary.

```
code:- import json
       with open("student.json", "r") as file:
           data = json.load(file)
       print(data)
```

JSON
{ "name": "Ramesh",
  "age": 22
}

o/p:-
{'name': 'Ramesh', 'age': 22}

→ (4) loads() :- Converts a JSON string into a python object.

```
import json
text = '{"name": "Ramesh", "age": 22}'
data = json.loads(text)
print(data)
```

o/p:-
{'name': 'Ramesh', 'age': 22}

* Conversions:

Python	dict	list	tuple	str	int	float	True	False	None
JSON	object	array	array	string	number	number	true	false	null

① Pretty Print :- Formats JSON to make it easier to read.
→ used for dump() & dumps() by passing indent parameter to json.dump().
→ Indent value (typically 2 (or) 4).

same eg;

```
result = json.dumps(student, indent=4)
print(result)
```

o/p:-
```
{
    "name": "Ramesh",
    "age": 22,
    "course": "AI/ML"
}
```
in more Readable form.

* Sort JSON keys :- add sort_keys=True parameter.

```
print(json.dumps(student, sort_keys=True, indent=4))
```

o/p:-
```
{
    "age": 22,
    "course": "AI/ML",
    "name": "Ramesh"
}
```
a, z → A → M (sorted order)

* JSON Encoding :- Converting Python object into JSON format
→ called as "Serialization".
eg:- dump(), dumps()

* JSON Decoding :- Converting JSON Data back to Python object.
→ called as "Deserialization".
eg:- load(), loads().

* Linear algebra :- Linear algebra is the mathematics of vectors & matrix.
→ It is used to represent & process data in Machine Learning & Deep Learning.

→ ① Vector:- Vector is an ordered collection of numbers, representing magnitude (&) direction.

Eg:- $V = [2, 4, 8]$ Length = 3

3-Dimensional vector.

operations:- ⊙ Addition
⊙ Subtraction
⊙ Dot product
⊙ Cross product
⊙ Magnitude

Applications:-
→ Images.
→ Recommendation systems.

→ ② ~~Magnitude~~ Matrix:- A rectangular arrangement of numbers in rows (&) columns.

Eg:- $A = \begin{vmatrix} 1 & 2 \\ 3 & 4 \end{vmatrix}$ ② 2 Rows, 2 columns

Operations:- Addition, Subtraction, Multiplication | Transpose, Inverse, Determinant

Application:
① Image processing
② Neural Networks.
③ Data storage.

→ ③ Eigen Vectors:- Eigen vector is a vector whose direction does not change, after multiplication by a matrix. only length changes.

Eg:- original (→) After transformation (——→).
Length changes, not direction.

→ ④ Eigen Value:- Value which tells, how much an eigen vector streches after matrix transformation.

$$A \times V = \lambda \times V$$

A: matrix, V: eigen vector, λ: eigen value (lambda), V: eigen vector.

classmate
Date
Page 15

② Probability (&) Statistics:- Probability helps us measure uncertainty, statistics helps us analyse data.

→ ① (i) Mean:- average of all values. $\text{mean} = \frac{\text{Sum of all values.}}{\text{Total no of values.}}$

② (ii) Median:- middle value after arranging data in ascending order.
If odd values, middle value is median.
even values, average of middle values is median.

③ (iii) ~~Median~~ Mode:- Value that occurs most frequently.

④ (iv) Variance:- It measures how far the data values are spread from the mean.

$$\text{Var}(X) = E(X^2) - [E(X)]^2$$

Low variance → Data points are close together.
high variance → " " " widely spread.

⑤ (v) Standard Deviation (SD):- average spread of data from mean.

Simply, $\sqrt{\text{variance}}$. ; $\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$

Low SD → Values are close to mean
High SD → Values are spread out.

29-07-2026 (11:00 AM) ① Vector addition
(ii) Vector multiplication [when we use we are using scalar multiplication in real life]

(iii) Dot Product:- Used similarity b/w the vectors. Dot product
Used in recommendations; search engines
(You can be next).

→ Anti Patterns (missing context, asking so many questions from prompt) → No success
→ Prompt Iteration loop.

H.W :- write with Vauge prompts (&) Quality prompt.
→ Future of prompting.

29-07-2026 (class Notes + tasks assigned).

Purpose in ML

→ Vector Addition:- → Combine multiple informations (&) features.
→ updates weights during training.

Real Eg:- ① Recommendation systems
⊙ User preference (Vector) + new interst (Vector) = updated user profile.
(+)
② Google map navigation systems.
⊙ walking direction (Vector) + current location (Vector) = New location
(+)

→ Scalar multiplication:- → multiply every element of the vector by one number.
→ used in neural network computations.
→ Scale feature values.

Real Eg:- Image Brightness adjustment
→ Every pixel should multiply with a constant.

original:- [10, 20, 30]
Increase brightness x 2 ⇒ [20, 40, 60].

Definition:- Two vectors produce a single number (scalar).

→ Dot Product:- → Measure similarity blw 2 vectors.

Real Eg:- ① Recommendation (OTT's)
⊙ Net flix ⇒ user likes Action, Comedy, Sci-fi,
New movie contain Action, Comedy. (It Recommends).
Dot product → High (As, it compare New movie with user's preferences)

→ MNIST Digits:- Data set used to train handwriting recognition models.
contains:- ⊙ 70,000 hand written digits
⊙ Images are 28 x 28 pixcels.

Real life Eg:- ① ATM cheque Reading
② Postal code Recognisation.

Purpose in ML

→ Matrix multiplication: Neural Network's perform matrix multiplication at every layer.

①②③ Weights: a number that tells the model how important a feature is.

General Real life Eg:- To hire a SE for a python developer:

Candidate have, python → Excellent = 0.9
communication → Excellent = 0.8 } Weights
cricket → Bad = 0.1 } Features of Candidate

Real life Eg:

Input: A tiny image of 2×2 → $\begin{vmatrix} 10 & 20 \\ 30 & 40 \end{vmatrix}$

Weights: Weights of each pixel ⇒ $\begin{vmatrix} 0.2 & 0.8 \\ 0.5 & 1.0 \end{vmatrix}$

matrix multiplication ⇒ $\begin{vmatrix} 10 & 20 \\ 30 & 40 \end{vmatrix} \times \begin{vmatrix} 0.2 & 0.8 \\ 0.5 & 1.0 \end{vmatrix}$ = output.

(Input) × (Weights) = output.

This output will be new input in another layer in neural network.

Real life Eg of Image Classification:-

Image Classification: upload a cat image.

↓

Neural network perform millions of matrix multiplication

↓

Predict "cat".

④ Vector multiplication:- ⑤ types (a) Dot product (b) Cross product.

(a) Cross product:- Produce another vector.

A × B = New Vector.

Used mainly in, Physics, Robotics, computer graphics.

→ Rarely used in ML.

→ Transpose Matrix (A^T):- ① Flip rows into columns.

② Required for matrix multiplication.

Real Eg:- → Suppose students are rows, subjects are columns.

① Sometimes, algorithms need, subject as rows and students as columns.

→ Tranpose perform this conversion.

→ Identity Matrix (I):- Acts like "1". Does not change the matrix.

Eg:	Real Eg:-
Used in:- ① Weight initialization. ② Numeric stability.	multiplying by identity matrix is like $I \cdot 500 \times = I \cdot 500$ (No change)

→ Inverse Matrix:- Inverse matrix helps to recover unknown values (x) helps in solving the equations in ML

→ Eigen Vector:- Shows the important direction of data.

Real Eg:- Face recognition.

Thousands of Pixels.

↓

Eigen vector find important facial patterns.

↓

Model recognizes faces.

→ Eigen Value:- Shows how much importance a direction has

Used in:- ① Image comparision ② School Management ③ Face recognition

Real Eg:- Suppose students scores have 20 features.

School management Keeps only 5 most important ones.

→ Eigen value decides, which feature matters most.

→ (3) Probability:- Handles uncertainity, used for predictions.

Real Eg:- "Spam Detection"

If, Email has win money, lottery, Free

probability, Spam = 98% model predicts, Spam

→ (4) Bayesian Learning:- updates prediction, whenever new information arrives.

Real Eg:- Medical diagnosis.

Initially, Disease probability → 20%

After, Lab reports, probability becomes → 90%

→ (5) Numpy:- used for "Fast numerical calculations."

→ works with Arrays (&) matrices.

common functions:-

1. np.array()
2. np.dot()
3. np.mean()

Real Eg:- 28 x 28 pixcels.

↓

Numpy Array

↓

Easy mathematical operations

→ (6) Pandas:- Load and Preprocess datasets.

Real Eg:

customer CV; Name

Age

Salary

Pandas:-

- Reads data
- Removing missing values.
- Filters rows.

→ (7) Scikit-learn:- provide ready-to-use ML algorithms.

Real Eg:- predict house prices;

Data

↓

Linear Regression

↓

Train

↓

predict